

CUDA. Laboratorium 1.

Maj-czerwiec 2026

Przygotowanie środowiska

W tym roku ćwiczenia z laboratorium wykonujemy w środowisku Google Colab. Elementarne informacje o uruchamianiu programów CUDA C/C++ w środowisku Google Colaboratory można znaleźć w bardzo wielu miejscach. Obecnie mogę zaproponować:
<https://medium.com/@zubair09/running-cuda-on-google-colab-d8992b12f767> (trzeba się zalogować np. przez google)
<https://moritzstrube.substack.com/p/how-to-run-cuda-in-a-colab-notebook>
<https://www.geeksforgeeks.org/how-to-run-cuda-c-c-on-jupyter-notebook-in-google-colaboratory/>
ale jest ich więcej.

W skrócie:

wchodzimy w przeglądarce na <https://colab.research.google.com>, wybieramy środowisko wykonawcze GPU – T4, sprawdzamy dla porządku wersje Pythona i CUDY, instalujemy mały dodatek do nvcc i uruchamiamy go w komórce notatnika:

```
!python --version
!nvcc --version
!pip install nvcc4jupyter
%load_ext nvcc4jupyter
```

Do uruchomienia mamy jeden program, który dodaje długie wektory w sposób równoległy. Naszym zadaniem będzie zmiana wskazanych parametrów zrównoleglania wektorów i zbadanie ich wpływu na czas wykonania.

Możemy na początek sprawdzić bardzo krótką charakterystykę środowiska za pomocą paru linijek skryptu w Pythonie:

```
import torch

print("Czy CUDA jest dostępna:", torch.cuda.is_available())
print("Liczba dostępnych GPU:", torch.cuda.device_count())
if torch.cuda.is_available():
    print("Nazwa GPU:", torch.cuda.get_device_name(0))
```

Oceńmy też jaka jest wartość Compute Capability w naszym środowisku. Można to zrobić za pomocą komendy `nvidia-smi` w formie:

```
!nvidia-smi --query-gpu=name,compute_cap --format=csv
```

Powinniśmy uzyskać informację o procesorze (Tesla T4) i wartość Compute Capability 7.5.

Następnie kompilujemy kod źródłowy np. komendą:

```
!nvcc add_vec_gpu.cu -o add_vec_gpu -arch=sm_75
```

Dodanie parametru z właściwą architekturą `sm_75` pozwala uniknąć ostrzeżeń, a czasem i błędów kompilacji.

Pomiar czasu

Pomiar czasu jest dla nas istotny, bo będzie podstawowy, jeśli nie jedyny wynik działającego przykładu.

Można go mierzyć na różne sposoby. Tutaj proponowane są dwa

Pierwszy sposób z `gputimer.h`...

Wykorzystujemy plik `gputimer.h` (załączony w przykładach), który jest dołączony do kodu `add_vec_gpu.cu`.

W kodzie programu piszemy np.:

```
GpuTimer = timer;
...
...
timer.Start();
//    wywołanie kernela
timer.Stop();
...
...
printf("..... timer.Elapsed()...); //wynik w milisekundach
```

Drugi sposób - wykorzystanie funkcji zdarzeń `cudaEvent_t()`

Ten pomiar jest rekomendowany jako bardziej precyzyjny. Trudno to ocenić – istotnie daje nieco inne wyniki niż `gputimer`. Został użyty w kodzie `add_vec_gpu_evtime.cu`.

Do obliczeń można wybrać dowolny z timerów. Gdyby zaś ktoś zrobił obliczenia szybko, może porównać oba.

Ćwiczenie 1(i jedyne): dodawanie wektorów.

Do ćwiczenia używamy jednego z dwóch programów: `add_vec_gpu.cu` lub `add_vec_gpu_evtime.cu`.

Proszę go skompilować i uruchomić, a następnie trochę go zmodyfikować.

- Łatwo zauważyć, że pomiary czasu w poszczególnych przebiegach charakteryzują się dużym rozrzutem. Proszę dołożyć pętlę, która w jednym uruchomieniu będzie wykonywała dodawanie wektorów np. 10 lub 20 razy, a następnie wyliczała i wyprowadzała uśredniony czas.
- Proszę dołożyć pętlę lub parametr do uruchomienia programu, który pozwalałby zmieniać rozmiar bloku. Zakładamy przy tym, że rozmiar wektora jest w tym momencie ustalony i wynosi $2^{20} = 1048576$ elementów. Rozmiar bloku powinien się zmieniać od 1 do 1024 wątków (zapewne najpierw gęściej, potem rzadziej). Wyniki powinny posłużyć do sporządzenia wykresu, w którym na osi x jest rozmiar bloku (czyli liczba wątków w bloku), a na osi y czas obliczeń w milisekundach. Czy zauważamy w wykresie jakieś minimum lub maksimum? Czy osie powinny być liniowe czy raczej logarytmiczne?

- Dla wybranego rozmiaru bloku (powiedzmy takiego przeciętnego) proszę przygotować wersję programu, która w pętli policzy czasy dodawania dla różnych długości wektora: od niewielkich, aż do wartości 2^{20} (właściwie można próbować jeszcze dalej). Ten wynik powinien stanowić podstawę drugiego wykresu, gdzie na osi x jest długość wektora, a na osi y – czas dodawania.
- Proszę dodatkowo sprawdzić, czy program daje poprawne wyniki. W szczególności co się stanie, gdy liczba wątków w bloku przekroczy 1024.

W ramach ćwiczenia proszę przysłać zmodyfikowane kody oraz dwa wykresy nadające się do oglądnięcia. Proszę też dołączyć króciutki komentarz dotyczący ostatniego punktu (przekroczenia granicy bloku 1024).